



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



MODULE 4

INTERMEDIATE CODE GENERATION

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Switch Case Statements

- The “case” or “switch” statement is available in various languages.

```
switch (E)
```

```
{
```

```
    case V1: S1
```

```
    case V2: S2
```

```
    ....
```

```
    case Vn-1: Sn-1
```

```
    default: Sn
```

```
}
```

- E** is a *selector expression*, which is to be evaluated, followed by n *constant values* ***V_i***, ***V₂***, ..., ***V_n*** that the expression might take, perhaps including a *default "value,"* which always matches the expression if no other value does.

Translation of Switch-Statements

- The intended translation of a switch is code to:
 - Evaluate the expression E*
 - Find value V_j in the list of cases that is same as the value of the expression*
 - Execute the statement S_j associated with the value found*
- Step (2) implemented as
 - Sequence of conditional jumps** (If the number of cases is small, say 10)
 - Create table of pairs** (value , label) for corresponding statement's code.
 - Hash table for the values** (If the number of values exceeds 10)

Syntax-Directed Translation of Switch-Statements

Code to evaluate E into t

goto **test**

L_1 : code for **S_1**
 goto **next**

L_2 : code for **S_2**
 goto **next**

.....

L_{n-1} : code for **S_{n-1}**
 goto **next**

L_n : code for **S_n**
 goto **next**

test: if $t = 1$ goto **L_1**
 if $t = 2$ goto **L_2**

.....

 if $t = V_{n-1}$ goto **L_{n-1}**
 goto **L_n**

next:

- When the **switch** keyword is seen then, two labels **test** and **next** and a temporary variable **t** are generated.
- When we see the **case** keyword, a new label **L_i** is created and entered into the symbol table.
- The value of **V_i** of each case constant and a pointer to this symbol-table entry are placed on a stack.

Reading the queue of value-label pairs, we can generate a sequence of three address statement as shown below.

```
case t  $V_1$   $L_1$ 
case t  $V_2$   $L_2$ 
...
case t  $V_{n-1}$   $L_{n-1}$ 
case t t  $L_n$ 
next:
```

- **t** is the temporary holding the value of the selector expression E
- **L_n** is the label for the default statement.
- The case **t V_i L_i** instruction is a synonym for if
 $t = V_i$ goto L_i

Example:

C Program

```
#include <stdio.h>
int main() {
    int day = 2;

    switch (day) {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        default:
            printf("Wrong option");
    }
    return 0;
}
```

Three address code

```
1: day = 2
2: goto test
// Case 1
3: t1 = "Monday"
4: call printf, t1
5: goto next
// Case 2
6: t2 = "Tuesday"
7: call printf, t2
8: goto next
// Case 3
9: t3 = "Wednesday"
10: call printf, t3
11: goto next
// Default case
12: t4 = " Wrong option "
13: call printf, t4
```

```
// Test label
```

```
test:
```

```
14: if day == 1 goto 3
```

```
15: if day == 2 goto 6
```

```
16: if day == 3 goto 9
```

```
17: goto 12
```

```
next:
```

```
18: return 0
```